

Historical Shipment Store

Infrastruktur penyimpanan data pengiriman: mencatat setiap prediksi rute sebagai shipment record dan menyediakan mekanisme pelaporan hasil aktual, sebagai fondasi data historis nyata yang sebelumnya tidak ada di sistem.

Branch: fix/hackathon-readiness

Service: services/ai/route-planner

Tanggal dokumentasi: 22 Agustus 2026

Terkait: PROPOSAL_ALIGNMENT_CHECKLIST — item No. 4
(Historical Shipment) & No. 8 (Feature Store)

1. Latar Belakang & Masalah

Audit kesesuaian proposal (`PROPOSAL_ALIGNMENT_CHECKLIST`) menemukan bahwa backend JaGOOD **tidak memiliki penyimpanan data pengiriman sama sekali**. Satu-satunya tabel Postgres yang ada sebelumnya, `bmkg_cache` , hanya berfungsi sebagai cache respons cuaca BMKG/Open-Meteo — bukan riwayat pengiriman.

Field `historical_delay_avg_hours` dan `historical_damage_rate` yang dipakai sebagai fitur model (di `app/services/historical_baseline.py`) murni formula deterministik (mode transportasi + jarak + tingkat eksposur), **bukan hasil pembelajaran dari data nyata**.

TUJUAN PEMBARUAN INI

Membangun infrastruktur penyimpanan agar mulai hari ini, setiap prediksi rute otomatis tercatat sebagai shipment record, dan hasil aktual di lapangan (keterlambatan sungguhan, apakah kargo rusak) dapat dilaporkan kembali terhadap record tersebut — sehingga data historis yang *benar-benar nyata* mulai terkumpul dari waktu ke waktu.

BATASAN YANG DISENGAJA

Ini murni infrastruktur. Tidak ada backfill data lama (memang tidak tersedia), tidak ada pipeline pelatihan ulang model otomatis, dan tidak ada UI frontend untuk menjelajah riwayat — seluruhnya backend/API sesuai keputusan scope bersama pengguna.

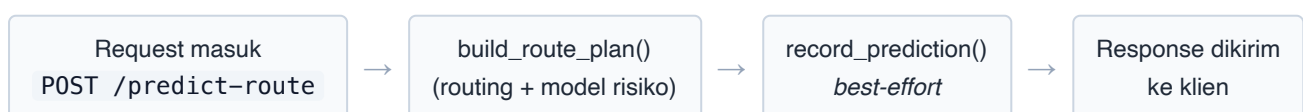
2. Keputusan Desain

2.1 Satu tabel lebar (`shipment_records`), bukan ternormalisasi

Kolom-kolom yang sering difilter (komoditas, moda transportasi, tanggal keberangkatan, level risiko) diratakan sebagai kolom biasa. Detail lengkap (rute alternatif, faktor SHAP, profil suhu kargo, geometri peta) disimpan sebagai satu kolom `JSONB` bernama `prediction_snapshot` . Pendekatan ini konsisten dengan gaya `bmkg_cache` yang sudah ada (satu `Table SQLAlchemy Core`, tanpa ORM/migration tool), dan tidak memerlukan JOIN untuk kedua endpoint baca.

2.2 Penulisan terjadi di router, bukan di `planning_service`

Pemanggilan penyimpanan diletakkan di handler `POST /predict-route` (`app/routers/route_planner.py`), **setelah** `build_route_plan()` berhasil — dibungkus `try/except` yang hanya mencatat log *warning*, tidak pernah menggagalkan response.



Alasan keputusan ini:

- **Isolasi kegagalan.** CI tidak menyediakan container Postgres untuk job `route-planner-tests` maupun `golden-demo`. Jika penulisan bersifat wajib (hard-fail), seluruh test `/predict-route` yang ada — termasuk golden demo — akan gagal begitu database tidak terjangkau. Audit-trail write memang seharusnya tidak pernah mematikan jalur prediksi inti.
- **Scenario simulation otomatis tidak tersimpan.** `simulate_scenario()` tidak pernah memanggil `build_route_plan()` — ia memanggil `evaluate_candidates()` langsung. Karena penulisan hanya ada di handler `/predict-route`, endpoint `/simulate-scenario` otomatis tidak mencatat apa pun ke `shipment_records`, tanpa perlu kode pengecualian tambahan — sesuai prinsip bahwa skenario "what-if" adalah hipotetis, bukan pengiriman nyata.

2.3 Status siklus hidup data: `outcome_status`

Mengikuti konvensi klasifikasi data yang sudah dipakai di repo ini (`REAL` / `REFERENCE` / `DERIVED` / `SYNTHETIC` / `DEMO` pada `commodity_provenance.json`), setiap baris shipment diberi status siklus hidup terpisah:

Nilai	Arti
<code>predicted_only</code>	Baris baru dibuat dari hasil prediksi model — belum ada konfirmasi hasil aktual di lapangan.
<code>outcome_reported</code>	Hasil aktual (delay & status kerusakan sungguhan) sudah dilaporkan lewat <code>PATCH /shipments/{id}/outcome</code> — titik pertama sistem memiliki data nyata (<code>REAL</code>) , bukan sintetis atau prediksi semata.

3. Skema Tabel `shipment_records`

Ditambahkan ke `app/core/db.py`, dibuat otomatis lewat `metadata.create_all(engine)` yang sudah ada (tidak perlu tool migrasi baru).

Kolom	Tipe	Keterangan
shipment_id	String (PK)	ID unik pengiriman, dari request atau digenerate otomatis
created_at	DateTime(tz)	Waktu record dibuat
— konteks permintaan —		
commodity_type	String, indexed	Jenis komoditas
origin_lat / origin_lon	Float	Koordinat asal
destination_lat / destination_lon	Float	Koordinat tujuan
departure_time	DateTime(tz), indexed	Waktu keberangkatan rencana
transport_mode_preference	String, nullable	Preferensi moda dari user (jika ada)
cold_chain_equipment	String	reefer / pasif
insulation_quality	String	baik / sedang / buruk
ranking_preference	String	risiko / kecepatan
— rute yang direkomendasikan (diratakan dari RouteCandidate) —		
recommended_route_id	String	ID rute yang direkomendasikan
transport_mode	String, indexed	Moda aktual rute terpilih
distance_km	Float	Jarak tempuh
estimated_duration_hours	Float	Estimasi durasi perjalanan
expected_delay_hours	Float	Delay tambahan yang diperkirakan (default 0)
estimated_arrival	DateTime(tz)	Estimasi waktu tiba
risk_level	String, indexed	Low / Medium / High
risk_probability	Float	Skor risiko kontinu (0–1)
confidence_score	Float	Keyakinan model pada prediksi teratas
historical_delay_avg_hours	Float	Baseline formula saat prediksi dibuat — dibekukan, tidak dihitung ulang
historical_damage_rate	Float	Baseline tingkat kerusakan saat prediksi dibuat
data_quality	String	Provenance routing: live / estimated
environmental_data_quality	String	Provenance cuaca/gelombang/suhu pelabuhan

<code>cargo_temperature_data_quality</code>	String	Provenance suhu kargo
— snapshot & hasil aktual —		
<code>prediction_snapshot</code>	JSONB	Seluruh response <code>PredictRouteResponse</code> (rute rekomendasi + alternatif) persis seperti yang dikirim ke klien
<code>outcome_status</code>	String, indexed	<code>predicted_only</code> (default) / <code>outcome_reported</code>
<code>actual_delay_hours</code>	Float, nullable	Delay aktual yang dilaporkan
<code>actual_damage_occurred</code>	Boolean, nullable	Apakah kargo benar-benar rusak
<code>outcome_notes</code>	Text, nullable	Catatan bebas terkait hasil aktual
<code>outcome_reported_at</code>	DateTime(tz), nullable	Waktu laporan hasil aktual diterima

4. Endpoint API Baru

POST `/predict-route` (sudah ada — kini otomatis mencatat shipment)

Tidak ada perubahan pada kontrak request/response. Setelah rute berhasil dibangun, sistem otomatis memanggil `shipment_service.record_prediction()` secara *best-effort* sebelum response dikembalikan ke klien.

GET `/shipments`

Daftar shipment record dengan filter dasar dan paginasi.

Query param	Tipe	Keterangan
<code>commodity_type</code>	string, opsional	Filter berdasarkan komoditas
<code>transport_mode</code>	string, opsional	Filter berdasarkan moda transportasi rute terpilih
<code>date_from</code> / <code>date_to</code>	datetime, opsional	Rentang <code>departure_time</code>
<code>limit</code>	int, default 50, maks 200	Jumlah per halaman
<code>offset</code>	int, default 0	Offset paginasi

```
{
  "items": [ { "shipment_id": "shp-...", "outcome_status": "predicted_only", ... } ],
  "total": 42,
  "limit": 50,
  "offset": 0
}
```

GET /shipments/{shipment_id}

Detail lengkap satu shipment record, termasuk `prediction_snapshot` penuh. Mengembalikan `404` jika `shipment_id` tidak ditemukan.

PATCH /shipments/{shipment_id}/outcome

Melaporkan hasil aktual pengiriman. Dipilih `PATCH` (bukan `POST`) karena ini satu-satunya endpoint yang memperbarui record yang sudah ada, bukan menghitung artefak baru.

```
// Request body
{
  "actual_delay_hours": 3.5,
  "actual_damage_occurred": false,
  "outcome_notes": "Tiba tepat waktu, kondisi kargo baik"
}
```

Efek: `outcome_status` berubah menjadi `outcome_reported`, `outcome_reported_at` diisi waktu sekarang, dan mengembalikan record lengkap yang sudah diperbarui. Mengembalikan `404` jika `shipment_id` tidak ditemukan.

5. File yang Diubah & Ditambahkan

[diubah] `services/ai/route-planner/app/core/db.py`

[diubah] `services/ai/route-planner/app/main.py`

[diubah] `services/ai/route-planner/app/routers/route_planner.py`

[baru] `services/ai/route-planner/app/schemas/shipment_schema.py`

[baru] `services/ai/route-planner/app/services/shipment_service.py`

[baru] `services/ai/route-planner/app/routers/shipments.py`

[baru] `services/ai/route-planner/tests/test_shipment_store_e2e.py`

6. Pengujian & Hasil Verifikasi

6.1 Dua lapis strategi tes

1. **Tanpa DB (berjalan di CI default):** membuktikan penulisan best-effort tidak pernah menggagalkan `/predict-route` walau `DATABASE_URL` tidak terjangkau; juga membuktikan `/simulate-scenario` tidak pernah memanggil `insert_shipment_record`; serta memverifikasi respons `404` yang benar untuk `shipment_id` tak dikenal.
2. **Dengan Postgres nyata (di-skip otomatis jika DB tak terjangkau):** round-trip penuh `predict-route` → `get shipment` → `patch outcome` → `get lagi`, filter `GET /shipments`, dan konfirmasi `simulate-scenario` tidak menambah baris.

6.2 Hasil aktual dijalankan

Suite	Kondisi	Hasil
Seluruh test suite <code>services/ai/route-planner/tests/</code> (36 test)	Tanpa Postgres live	34 passed, 2 skipped (dua tes yang memang butuh DB nyata)
<code>tests/golden_demo/test_golden_demo.py</code>	Tanpa Postgres live	1 passed — tidak ada perubahan kode diperlukan pada golden demo
<code>test_shipment_store_e2e.py</code> (6 test)	Dengan Postgres live (<code>docker compose up -d postgres</code>)	6 passed — termasuk round-trip persistensi nyata

KESIMPULAN VERIFIKASI

Fitur ini tidak mengubah perilaku endpoint yang sudah ada dan tidak menambah dependensi wajib terhadap Postgres pada jalur test/CI yang berjalan tanpa database.

7. Keterhubungan dengan Rencana Selanjutnya

Setelah baris `shipment_records` dengan `outcome_status = "outcome_reported"` cukup terkumpul, langkah lanjutan (belum dibangun saat ini) adalah membuat batch job yang mengagregasi `actual_delay_hours` / `actual_damage_occurred` berdasarkan kombinasi (`transport_mode`, rentang jarak, tingkat eksposur) — kunci koridor yang sama yang sudah dipakai `training/synthetic_corridors.py` — lalu memasukkan agregat nyata tersebut sebagai lookup di depan formula deterministik `historical_baseline.py`, atau menggantikan prior sintetis pada training run `train_model.py` berikutnya. Tidak ada perubahan skema tambahan yang dibutuhkan untuk langkah ini — `prediction_snapshot` dan kolom `historical_*` yang sudah dibekukan pada saat prediksi sudah cukup untuk membandingkan "apa yang diprediksi formula" dengan "apa yang benar-benar terjadi".